TEAM REVIEW EDITION

WS-AUTH-001: Workstream Actor Profile, Role, and Authorization Service Specification

A locked implementation-handoff specification prepared for structured team review.

DOCUMENT	WS-AUTH-001
STATUS	Locked for implementation handoff
MATURITY	Design complete; not yet runtime-proven
VERSION / DATE	0.1 / 2026-07-11
OWNER	Workstream Engineering
SCOPE	Identity Issuer token verification, Workstream ActorProfile provisioning, identity linking, contributor-domain membership, admin-role grants, project-role grants, permission evaluation, resource guards, revocation, bootstrap, audit, APIs, database constraints, concurrency, conformance, and live proof

Review instruction: validate domain decisions, invariants, transaction boundaries, failure behaviour, and conformance coverage. Proposed changes should identify the exact section and affected invariant.

Contents

Use the PDF outline or the section index below to navigate the specification.

1. Purpose

This specification defines the Workstream authority spine that MUST exist before the review lifecycle is implemented.

It establishes:

- how an Identity Issuer token is verified;
- how one issuer subject resolves to exactly one local Workstream ActorProfile;
- how a first-time human caller is provisioned into the Contributor domain;
- why Contributor membership does not grant project work authority;
- how project-scoped submitter and reviewer privileges are granted;
- how administrative roles are granted independently of contributor privileges;
- the five initial Workstream administrator roles;
- the exact permission catalog for those roles;
- how roles, project scope, resource ownership, resource state, and explicit guards combine into one authorization decision;
- how suspension, deactivation, role revocation, and identity-link revocation take effect;
- how bootstrap authority is established without a hidden database edit;
- which APIs, constraints, audit events, tests, and live proof are required.

The objective is to ensure that every later Workstream feature calls one established authorization service instead of implementing identity or permissions inside individual routers.

---

2. Normative Relationship to Other Specifications

This specification becomes the authority source for ActorProfile, identity links, admin roles, contributor project roles, and authorization evaluation.

The following precedence applies:

1. `WS-AUTH-001` controls identity resolution, ActorProfile state, `AdminRoleGrant`, `ProjectRoleGrant`, role scope, permission evaluation, and authority revocation.
2. `WS-REV-001` controls review queue routing, leases, review decisions, revision chains, and review-specific resource guards.
3. `WS-CON-001` controls contribution creation, compensation policies, awards, adapters, and finance-specific resource behavior.
4. `WS-ARCH-001` controls the overall Workstream system boundary, container/component direction, evidence boundary, deployment, and operational architecture.

Where `WS-REV-001` currently defines `ProjectRoleGrant`, this specification supersedes that object only for creation authority, scope, persistence, and authorization behavior. The review specification continues to control how submitter/reviewer grants are consumed by task and review operations.

No implementation of `WS-REV-001` may begin until the definition of done in this specification has been met.

---

3. Normative Language

The words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, and MAY are normative.

- MUST and SHALL describe mandatory v0.1 behavior.
- MUST NOT and SHALL NOT describe prohibited behavior.
- SHOULD describes a strong implementation recommendation that may be changed only without changing the contract.
- MAY describes an optional implementation detail.

4. Locked v0.1 Decisions

The coding agent is not permitted to reinterpret the following decisions.

1. Identity Issuer owns portable identity, final token signing, and the public product-trusted JWKS.
2. Workstream owns ActorProfiles, roles, permissions, project grants, resource guards, and authorization decisions.
3. Identity Issuer tokens do not contain authoritative Workstream roles.
4. The stable external identity anchor is `(issuer, subject)`.
5. One `(issuer, subject)` maps to exactly one Workstream ActorProfile.
6. One Workstream ActorProfile has exactly one active ActorIdentityLink in v0.1.
7. First-time valid human callers are automatically provisioned as active human ActorProfiles.
8. Every active human ActorProfile belongs to the Contributor domain by default.
9. Contributor-domain membership is not task, submission, or review authority.
10. Project work requires an active `ProjectRoleGrant` for that exact project.
11. Project contributor roles are `submitter`, `reviewer`, or `both`.
12. Project roles are explicitly granted by an authorized Project Manager in v0.1.
13. Skills and reputation may inform a ProjectRoleGrant but do not automatically create one.
14. Administrative authority is represented by separate `AdminRoleGrant` records.
15. An ActorProfile may hold multiple AdminRoleGrants.
16. An ActorProfile may be both an administrator and a project contributor.
17. Administrative authority never automatically grants submitter or reviewer authority.
18. An administrator who submits or reviews work must hold the same active ProjectRoleGrant required of any contributor.
19. The initial administrator roles are `access_administrator`, `operator`, `project_manager`, `finance_authority`, and `audit_authority`.
20. Roles grant permission candidates; resource scope and resource guards still decide the command.
21. Authorization is deny-by-default.
22. An explicit resource guard overrides role-derived permission.
23. Role and grant revocation take effect from Workstream state without waiting for a new Identity Issuer token.
24. Authorization decisions are not cached across requests in v0.1.
25. Service subjects are never automatically provisioned.
26. Unknown service subjects must be pre-provisioned by an Access Administrator.
27. Agent and Space subjects are outside the v0.1 Workstream access contract and are denied.

28. No user can grant themselves an AdminRoleGrant or ProjectRoleGrant.
29. The last active Access Administrator cannot be revoked, suspended, or deactivated.
30. No authorization record is deleted. Revocation and deactivation preserve history.

5. Authority Model

```
Identity Issuer
  issuer + subject + signed Workstream token
                  |
                  v
ActorIdentityLink (one external identity anchor)
                  |
                  v
ActorProfile (one local Workstream actor)
        |
        +-- Human actor => Contributor domain by default
        |       |
        |       +-- ProjectRoleGrant(project A, submitter)
        |       +-- ProjectRoleGrant(project B, reviewer)
        |       +-- ProjectRoleGrant(project C, both)
        |
        +-- Zero or more AdminRoleGrants
                +-- access_administrator
                +-- operator
                +-- project_manager
                +-- finance_authority
                +-- audit_authority
```

The same ActorProfile is used everywhere in Workstream. The system MUST NOT create a contributor profile and a second administrator profile for the same issuer subject.

6. Core Object Model

6.1 ActorProfile

`ActorProfile` is Workstream's local identity for a human or service actor.

```
ActorProfile:
  id: uuid
  actor_kind: enum [human, service]
  status: enum [active, suspended, deactivated]
  provisioning_method: enum [automatic_first_access, manual_service_provisioning]

  display_name: string | null
  contact_email: string | null

  created_by: actor_id | system_actor_id
  created_at: timestamp
  updated_at: timestamp
  last_seen_at: timestamp | null

  suspended_by: actor_id | null
  suspended_at: timestamp | null
  suspension_reason: string | null

  deactivated_by: actor_id | null
  deactivated_at: timestamp | null
  deactivation_reason: string | null
```

ActorProfile invariants

- `actor_kind` is immutable.

- `provisioning_method` is immutable.
- A first-time human token creates `actor_kind = human` and `provisioning_method = automatic_first_access`.
- A service actor is created only by an Access Administrator and uses `manual_service_provisioning`.
- `display_name` and `contact_email` are profile/display data and MUST NOT affect authorization.
- `suspended` is reversible.
- `deactivated` is terminal in v0.1.
- ActorProfile rows are never deleted.
- Historical submissions, reviews, contributions, awards, audit events, and grants remain linked after suspension or deactivation.
- Every active human ActorProfile is a Contributor-domain member by definition. Contributor membership is derived from `actor_kind = human`; it is not a separate mutable role grant.
- An ActorProfile is considered part of the Admin domain only when it has at least one effective active AdminRoleGrant.

6.2 ActorIdentityLink

`ActorIdentityLink` connects the stable Identity Issuer anchor to one ActorProfile.

```
ActorIdentityLink:
  id: uuid
  actor_profile_id: uuid
  issuer: string
  subject: string
  subject_kind: enum [human, service]
  status: enum [active, revoked]

  linked_by: actor_id | system_actor_id
  linked_at: timestamp
  last_verified_at: timestamp

  revoked_by: actor_id | null
  revoked_at: timestamp | null
  revoked_reason: string | null

  reactivated_by: actor_id | null
  reactivated_at: timestamp | null
  reactivation_reason: string | null
```

ActorIdentityLink invariants

- `issuer` is stored in canonical normalized URL form.
- `subject` is opaque and case-sensitive.
- `UNIQUE(issuer, subject)` is mandatory.
- `UNIQUE(actor_profile_id)` is mandatory in v0.1.
- The link `subject_kind` must match ActorProfile `actor_kind`.
- Human first access creates the ActorProfile and ActorIdentityLink in one transaction.
- Workstream does not merge Identity Issuer subjects.
- Adding multiple human subjects to one ActorProfile is out of scope for v0.1.
- Revocation prevents the linked subject from using the ActorProfile.
- Reactivation restores the same link; it does not create another ActorProfile.
- Revoked links are never deleted or replaced with a new row for the same `(issuer, subject)`.

6.3 AdminRoleGrant

`AdminRoleGrant` represents additive Workstream administrative authority.

```
AdminRoleGrant:
  id: uuid
  actor_profile_id: uuid
  role: enum [
    access_administrator,
    operator,
    project_manager,
    finance_authority,
    audit_authority
  ]

  scope_type: enum [system, project]
  scope_project_id: uuid | null
  status: enum [active, revoked]

  granted_by: actor_id | system_actor_id
  granted_by_admin_role_grant_id: uuid | null
  granted_at: timestamp
  grant_reason: string

  revoked_by: actor_id | null
  revoked_at: timestamp | null
  revoked_reason: string | null
```

AdminRoleGrant scope compatibility

Role	Allowed scopes
`access_administrator`	`system` only
`operator`	`system` only
`project_manager`	`system` or `project`
`finance_authority`	`system` or `project`
`audit_authority`	`system` or `project`

AdminRoleGrant invariants

- `scope_project_id` is null if and only if `scope_type = system`.
- A project-scoped grant applies to exactly one project.
- A system-scoped grant applies to all resources allowed by that role.
- Role grants are additive; effective permissions are the union of active grants whose scopes cover the resource.
- Explicit resource guards and ActorProfile state override the union.
- An actor cannot grant an AdminRoleGrant to themselves.
- Only an effective Access Administrator may issue or revoke an AdminRoleGrant.
- An Access Administrator may not grant a role outside the scope compatibility table.
- An actor cannot revoke their own AdminRoleGrant.
- The final effective active `access_administrator` grant cannot be revoked.
- Changing role or scope requires a new grant and revocation of the old grant. The existing row is not overwritten.
- Revoked grants remain permanent audit history.

6.4 ProjectRoleQualificationSnapshot

The qualification snapshot records the evidence visible when a Project Manager makes a manual contributor-role decision.

```
ProjectRoleQualificationSnapshot:
  id: uuid
  project_id: uuid
  contributor_id: uuid

  skills_snapshot: object
  reputation_snapshot: object
  prior_project_work_refs: list[uuid]
  external_expertise_refs: list[string]

  captured_by: actor_id
  captured_at: timestamp
```

Qualification snapshot invariants

- The snapshot is immutable.
- Missing skills or reputation are recorded explicitly as unavailable.
- Missing reputation does not block a manual v0.1 grant.
- The snapshot is evidence for the grant decision, not an authorization decision by itself.
- Snapshot payloads MUST exclude secrets and unnecessary personal information.

6.5 ProjectRoleGrant

`ProjectRoleGrant` is the only contributor-side project authority in v0.1.

```
ProjectRoleGrant:
  id: uuid
  project_id: uuid
  contributor_id: uuid
  role: enum [submitter, reviewer, both]
  status: enum [active, revoked]

  grant_method: enum [manual, automated]
  automation_policy_ref: uuid | null
  qualification_snapshot_ref: uuid

  granted_by: actor_id
  granted_by_admin_role_grant_id: uuid
  granted_at: timestamp
  grant_reason: string

  revoked_by: actor_id | null
  revoked_at: timestamp | null
  revoked_reason: string | null
```

ProjectRoleGrant invariants

- The contributor must be an active human ActorProfile.
- The grant is valid for one project only.
- A Project Manager whose effective scope covers the project may issue or revoke it.
- The issuing Project Manager cannot grant a ProjectRoleGrant to themselves.
- `grant_method = manual` is the only creation path enabled in v0.1.
- `grant_method = automated` is schema-reserved and MUST NOT be emitted by v0.1 code.
- `automation_policy_ref` is null for a manual grant.

- `qualification_snapshot_ref` is mandatory and must match the same project and contributor.
- At most one active ProjectRoleGrant exists for a contributor in a project.
- Changing `submitter`, `reviewer`, or `both` requires revoking the active grant and creating a new grant in one transaction.
- A submitter action requires `role IN (submitter, both)`.
- A reviewer action requires `role IN (reviewer, both)`.
- AdminRoleGrant does not satisfy either requirement.
- Revocation does not delete or invalidate historical work completed while the grant was active.

6.6 AuthorizationContext

`AuthorizationContext` is an in-memory request object. It is not a canonical database record.

```
AuthorizationContext:
  request_id: uuid
  correlation_id: uuid
  token_issuer: string
  token_subject: string
  token_id: string
  token_scopes: list[string]

  actor_profile_id: uuid
  actor_kind: enum [human, service]
  actor_status: enum [active, suspended, deactivated]

  active_admin_role_grants: list[AdminRoleGrantSummary]
  active_project_role_grants: list[ProjectRoleGrantSummary]
```

The context may be reused inside one request only. It MUST NOT be cached for authorization of a later request.

6.7 AuthorizationDecision

`AuthorizationDecision` is the result returned by the authorization service.

```
AuthorizationDecision:
  allowed: boolean
  permission: string
  actor_profile_id: uuid
  resource_type: string
  resource_id: uuid | null
  project_id: uuid | null
  matched_grant_ids: list[uuid]
  denial_code: string | null
  evaluated_at: timestamp
```

The full decision is written as an audit fact only for sensitive operations and configured denial classes. Normal request logs may contain the decision code and identifiers but MUST NOT contain token material.

6.8 AuthorityControl

`AuthorityControl` is a singleton PostgreSQL row used to serialize bootstrap and final-Access-Administrator safety checks.

```
AuthorityControl:
  id: integer # fixed value 1
  version: integer
  updated_at: timestamp
```

The bootstrap operation and every AdminRoleGrant/ActorProfile transition that could reduce the effective Access Administrator count MUST lock this row with `FOR UPDATE` before counting effective Access Administrators and committing the transition.

7. Identity Issuer Token Contract

7.1 Trusted issuer boundary

Workstream trusts only final tokens signed by the Identity Issuer-owned keys.

It MUST NOT trust:

- Better Auth JWT/JWKS output;
- a client-generated token;
- a token signed by Workstream;
- a token forwarded from another product without the Workstream audience;
- identity claims supplied in request JSON or headers outside the bearer token.

7.2 Required claims

A Workstream token MUST contain:

```
iss: canonical Identity Issuer URL
sub: opaque portable subject
aud: contains workstream
exp: expiry timestamp
iat: issued-at timestamp
jti: unique token identifier
subject_kind: enum [human, service, agent, space]
scope: space-delimited string or equivalent list
```

If the Identity Issuer exposes the subject classification under another approved canonical claim name, TokenVerifier may normalize that verified claim to `subject_kind` inside `VerifiedIssuerToken`. Workstream MUST NOT infer a missing classification from email, subject prefix, request route, or client input, and MUST NOT default an unknown subject to human.

Required deployment defaults:

```
WORKSTREAM_TOKEN_AUDIENCE=workstream
WORKSTREAM_REQUIRED_HUMAN_SCOPE=workstream:access
WORKSTREAM_REQUIRED_SERVICE_SCOPE=workstream:service
```

Coarse token scope only permits the request class to reach Workstream. It never replaces local authorization.

7.3 Supported subject kinds

`subject_kind`	**v0.1 behavior**
`human`	Auto-provision on first valid access
`service`	Resolve only if pre-provisioned by Access Administrator
`agent`	Deny with `unsupported_subject_kind`
`space`	Deny with `unsupported_subject_kind`

Delegated agent access is a future additive specification. The coding agent MUST NOT treat an agent token as a human contributor.

7.4 Verification algorithm

The TokenVerifier MUST:

1. parse the bearer token without trusting unverified claims;
2. select only configured Identity Issuer algorithms;
3. resolve `kid` against the issuer-owned JWKS;
4. verify signature;
5. validate exact canonical `iss`;
6. validate Workstream audience;
7. validate `exp`, `iat`, and `nbf` when present using configured clock skew;
8. require non-empty `sub`, `jti`, and `subject_kind`;
9. validate the required coarse scope for human or service access;
10. apply the configured introspection/revocation rule when required;
11. return verified claims to ActorResolver.

Unknown `kid` triggers one controlled JWKS refresh before failure. It MUST NOT disable signature verification.

7.5 JWKS caching

- JWKS may be cached according to issuer cache headers and configured maximum age.
- Key overlap must allow safe issuer rotation.
- Authorization roles and grants MUST NOT be stored in the JWKS cache.
- JWKS unavailability with no valid cached key returns `identity_verification_unavailable`.
- JWKS failure MUST NOT cause Workstream to accept an unverifiable token.

8. First Human Access and Automatic Provisioning

8.1 Preconditions

Automatic provisioning occurs only when:

- the token is fully verified;
- `aud` includes Workstream;
- required human scope is present;
- `subject_kind = human`;
- `(issuer, subject)` has no existing ActorIdentityLink.

8.2 Provisioning transaction

The first access transaction MUST:

1. normalize `issuer`;
2. attempt to load and lock an existing ActorIdentityLink;
3. create an active human ActorProfile if no link exists;

4. create the active ActorIdentityLink;
5. set `provisioning_method = automatic_first_access`;
6. copy only permitted display claims into `display_name` and `contact_email`;
7. record `ActorProfileProvisioned` and `ActorIdentityLinked` audit events;
8. commit;
9. return the ActorProfile as a Contributor-domain actor with no AdminRoleGrant and no ProjectRoleGrant.

8.3 Concurrent first access

If two first requests arrive for the same `(issuer, subject)`:

- exactly one ActorIdentityLink may be inserted;
- exactly one ActorProfile may remain associated with the link;
- the losing transaction must resolve the unique conflict by loading the committed profile;
- no orphan ActorProfile may remain.

The implementation SHOULD insert or reserve the identity link and profile in a transaction pattern that prevents orphan profiles. A cleanup job is not a substitute for transaction correctness.

8.4 First-access response

```
{
  "actor_profile_id": "uuid",
  "actor_kind": "human",
  "status": "active",
  "domains": ["contributor"],
  "admin_roles": [],
  "project_role_grants": []
}
```

The response MUST NOT imply that the actor can claim a task or review.

9. Service Actor Provisioning

Service actors are used for authenticated external adapters and approved Workstream service integrations.

9.1 Creation

An effective Access Administrator MUST create the service ActorProfile and ActorIdentityLink before the first service request.

Required request data:

```
{
  "issuer": "https://identity.flowresearch.tech",
  "subject": "sub_service_123",
  "display_name": "Project compensation adapter",
  "grant_reason": "Approved project fulfillment integration"
}
```

9.2 Service authority

- A service ActorProfile does not belong to the Contributor domain.
- A service ActorProfile cannot receive a ProjectRoleGrant.

- A service ActorProfile cannot receive an AdminRoleGrant in v0.1.
- Service authority is provided by explicit adapter binding or service-resource binding defined by the consuming specification.
- The authorization service must validate both the active service ActorProfile and the consuming binding.

9.3 Unknown service subject

A valid service token with no pre-provisioned link returns `403 service_actor_not_provisioned` and creates no ActorProfile.

---

10. Administrative Roles

10.1 Access Administrator

Purpose: manage Workstream actor and role authority.

May:

- read ActorProfile status and identity-link metadata;
- suspend eligible actors;
- reactivate suspended actors;
- deactivate eligible actors;
- revoke or reactivate identity links;
- provision service actors;
- issue and revoke AdminRoleGrants;
- read the permission catalog and role-grant audit history.

May not by this role alone:

- manage project guides, tasks, submissions, reviews, compensation, or awards;
- issue ProjectRoleGrants;
- submit or review work;
- view artifact content;
- grant themselves another role.

10.2 Operator

Purpose: operate and recover Workstream runtime behavior.

May:

- read system and project operational status;
- inspect task/review queue state across projects;
- run approved timer sweeps, reconciliation, outbox retry, and projection rebuild operations;
- force-release a review lease through the review specification's audited override;
- inspect operational audit events needed for recovery.

May not by this role alone:

- create or change AdminRoleGrants or ProjectRoleGrants;
- approve project policy;

- record a review decision;
- publish compensation policy;
- alter immutable records;
- mark an award fulfilled.

10.3 Project Manager

Purpose: own Workstream project configuration and project contributor authority.

May within effective scope:

- create projects when system-scoped;
- read and update project configuration;
- create and approve guide/effective policy versions;
- create, release, pause, or close tasks through project policy;
- configure review/revision policy;
- create qualification snapshots;
- issue and revoke ProjectRoleGrants;
- inspect project task and review queues;
- read project review and contribution history required for management.

May not by this role alone:

- grant AdminRoleGrants;
- grant themselves a ProjectRoleGrant;
- submit or review work;
- publish compensation policy;
- run system-wide recovery outside project-manager operations;
- alter immutable submissions, reviews, or contributions.

10.4 Finance Authority

Purpose: manage project compensation configuration and observe fulfillment.

May within effective scope:

- read project identity and contribution facts needed for compensation;
- create, edit draft, publish, and retire compensation policy versions;
- create, suspend, resume, and retire project adapter bindings;
- read CompensationAwards and fulfillment projections/receipts;
- initiate approved redelivery/reconciliation through the compensation specification.

May not by this role alone:

- create or alter Review decisions or ContributionRecords;
- mark fulfillment successful without an authenticated adapter receipt;
- manage ProjectRoleGrants or AdminRoleGrants;
- submit or review work;
- read unrelated project finance data.

10.5 Audit Authority

Purpose: read immutable and operational evidence without mutation.

May within effective scope:

- read audit events;
- read role/grant history;
- read task, submission, checker, review, contribution, award, and fulfillment chains;
- export authorized audit evidence;
- view minimal actor metadata required to attribute actions.

May not:

- execute any product mutation;
- reveal secrets, bearer tokens, artifact bytes beyond separately authorized evidence access, or unnecessary personal information;
- grant roles;
- run recovery;
- record review or fulfillment outcomes.

11. Permission Catalog

Permission identifiers are stable internal/public contract strings.

```
actor.profile.read_self
actor.profile.update_self
actor.profile.read_any
actor.profile.suspend
actor.profile.reactivate
actor.profile.deactivate
actor.identity_link.read
actor.identity_link.revoke
actor.identity_link.reactivate
actor.service.provision

admin_role.read
admin_role.grant
admin_role.revoke

project.create
project.read
project.update
project.archive
project.guide.manage
project.effective_policy.manage
project.task.manage
project.review_policy.manage
project.role_grant.read
project.role_grant.manage

task.queue.read
task.claim
submission.create
submission.read_own
submission.read_for_review

review.queue.read
review.queue.inspect
review.claim
review.release
review.decline_preference
review.decision
review.lease.force_release
review.chain.read

contribution.read_self
contribution.read_project

compensation.policy.manage
compensation.adapter_binding.manage
compensation.award.read
compensation.delivery.reconcile

operations.status.read
operations.timer.run
operations.reconcile.run
operations.outbox.retry
operations.projection.rebuild

audit.read
audit.export
```

Adding a permission identifier requires a new specification or an approved additive update. The coding agent MUST NOT invent permissions inside routers.

12. Permission Assignment Matrix

Legend: `S` = system scope only, `C` = covered system/project scope, `-` = not granted by this admin role.

Permission group	Access Administrator	Operator	Project Manager	Finance Authority	Audit Authority
Actor self permissions	inherited human behavior	inherited human behavior	inherited human behavior	inherited human behavior	inherited human behavior

Permission group	Access Administrator	Operator	Project Manager	Finance Authority	Audit Authority
Actor/profile administration	S	-	-	-	read-minimal C
Identity-link administration	S	-	-	-	read C
Service-actor provisioning	S	-	-	-	read C
Admin-role administration	S	-	-	-	read C
Project create	-	-	system-scoped grant only	-	-
Project read	-	S	C	C	C
Project update/archive	-	-	C	-	-
Guide/effective-policy management	-	-	C	-	read C
Task management	-	operational override only	C	-	read C
Review-policy management	-	-	C	-	read C
ProjectRoleGrant administration	-	-	C	-	read C
Review queue inspection	-	S	C	-	read C
Force-release review lease	-	S	-	-	-
Review decision	requires ProjectRoleGrant	requires ProjectRoleGrant	requires ProjectRoleGrant	requires ProjectRoleGrant	requires ProjectRoleGrant
Contribution project read	-	operational C	C	C	C
Compensation-policy management	-	-	-	C	read C
Adapter-binding management	-	-	-	C	read C
Award/receipt read	-	operational C	read C	C	C
Reconciliation/runtime operations	-	S	-	compensation-only C	-
Audit read/export	authority audit only S	operational audit S	project audit C	finance audit C	C

This matrix grants permission candidates. Every request still passes the authorization algorithm and resource guards.

13. Contributor Permission Matrix

13.1 Default human Contributor domain

An active human ActorProfile without any project grant may:

- use `actor.profile.read_self`;
- use `actor.profile.update_self` for permitted display fields;

- use `contribution.read_self` subject to record-level disclosure policy;
- accept project invitations or await an admin grant through future UI flow.

It may not claim tasks, create submissions, read reviewer queues, or review work.

13.2 Submitter grant

An active `ProjectRoleGrant(submitter|both)` supplies candidate permissions for the same project:

```
project.read
task.queue.read
task.claim
submission.create
submission.read_own
review.chain.read for the actor's own submitted task chain
```

TaskAssignment state, task ban, task availability, policy state, and other submitter resource guards still apply.

13.3 Reviewer grant

An active `ProjectRoleGrant(reviewer|both)` supplies candidate permissions for the same project:

```
project.read
review.queue.read
review.claim
review.release
review.decline_preference
review.decision
submission.read_for_review
review.chain.read
```

Self-review, active-lease capacity, preferred-reviewer routing, lease state, and decision-state guards still apply.

13.4 Both grant

`both` is the union of submitter and reviewer project permissions. It does not bypass self-review or any other resource guard.

---

14. Scope Resolution

14.1 System scope

A system-scoped AdminRoleGrant covers all projects only for permissions assigned to that role.

System scope is not superuser authority. For example, a system-scoped Finance Authority can manage compensation across projects but cannot record reviews.

14.2 Project scope

A project-scoped AdminRoleGrant covers only resources whose canonical `project_id` equals `scope_project_id`.

The authorization service MUST derive project ownership from canonical database relationships. It MUST NOT trust `project_id` supplied in a request body when the resource ID resolves to another project.

14.3 Cross-project relationships

If any related object belongs to a different project, authorization fails with `resource_project_mismatch`. Service validation does not replace database foreign-key ownership constraints.

15. Authorization Algorithm

Every protected command MUST call:

```
authorize(actor_context, permission, resource_context)
```

The authorization service MUST evaluate in this order:

1. Token was verified by TokenVerifier.
2. ActorIdentityLink exists and is active.
3. ActorProfile exists and is active.
4. Token `subject_kind` matches ActorProfile `actor_kind`.
5. Required coarse token scope is present.
6. Permission is a registered permission identifier.
7. Load active AdminRoleGrants whose scopes cover the resource.
8. Load the relevant active ProjectRoleGrant when the permission is contributor-side.
9. Determine whether at least one effective grant supplies the permission.
10. Resolve the resource's canonical project and ownership chain.
11. Apply global and resource-specific guards.
12. Apply resource-state preconditions.
13. Return allow or a stable denial code.

Pseudo-code:

```
def authorize(ctx, permission, resource):
    require_active_identity_link(ctx)
    require_active_actor(ctx)
    require_registered_permission(permission)

    candidates = permissions_from_effective_admin_grants(ctx, resource)
    candidates |= permissions_from_project_role_grant(ctx, resource)
    candidates |= default_human_self_permissions(ctx, resource)

    if permission not in candidates:
        deny("permission_not_granted")

    require_canonical_project_match(resource)
    apply_global_guards(ctx, permission, resource)
    apply_resource_guards(ctx, permission, resource)
    apply_state_preconditions(permission, resource)

    allow()
```

Routers and workers MUST NOT reproduce this algorithm independently.

16. Global Resource Guards

The following guards apply even when a role supplies the requested permission.

1. Suspended ActorProfile cannot execute business mutations.
2. Deactivated ActorProfile cannot execute any business command.
3. Revoked ActorIdentityLink cannot authenticate to the linked ActorProfile.
4. Service actors cannot receive human contributor permissions.
5. Human actors cannot use a service-only adapter binding.
6. An actor cannot issue a grant to themselves.
7. An actor cannot revoke their own AdminRoleGrant.
8. The final Access Administrator cannot be suspended, deactivated, or have the final grant revoked.
9. A reviewer cannot review their own submission.
10. A task-banned contributor cannot reclaim or resubmit that task.
11. A role whose scope does not cover the canonical project is ineffective.
12. A read operation must not reveal unauthorized project existence through counts or error differences.
13. Immutable records have no update/delete authorization path.

Consuming specifications may add narrower guards. They may not remove these guards.

---

17. Actor State Semantics

17.1 Active

An active actor may exercise effective permissions and grants.

17.2 Suspended

Suspension is a reversible security/operational control.

Effects:

- bearer token verification may succeed;
- ActorResolver returns the suspended ActorProfile;
- all business mutations are denied with `actor_suspended`;
- `/v1/actors/me` may return minimal status and support information;
- active AdminRoleGrants and ProjectRoleGrants remain recorded but ineffective;
- active review leases and other exclusive work claims are invalidated through `AuthorityInvalidationRequested` and consuming lifecycle reconciliation;
- historical records remain unchanged.

Reactivation restores still-active, non-revoked grants. It does not restore grants revoked separately during suspension.

17.3 Deactivated

Deactivation is terminal in v0.1.

Effects:

- all permissions and grants become permanently ineffective;
- all business APIs return `actor_deactivated` after identity resolution;
- identity link and grant history remain;

- consuming lifecycles reconcile active claims;
- the ActorProfile cannot be reactivated;
- a new ActorProfile MUST NOT be created for the same `(issuer, subject)`.

17.4 State-transition rules

```
active --suspend--> suspended --reactivate--> active
active --deactivate--> deactivated
suspended --deactivate--> deactivated
deactivated --X--> no transition
```

An Access Administrator cannot suspend or deactivate themselves. Another Access Administrator must execute the action.

18. Role and Identity Revocation

18.1 AdminRoleGrant revocation

Revocation:

- locks the target grant;
- verifies the actor is an effective Access Administrator;
- enforces no-self-revoke and last-access-admin guards;
- sets status/revocation fields;
- appends audit and `AuthorityInvalidationRequested` events;
- commits atomically.

The permission disappears on the next request because role decisions are not cached across requests.

18.2 ProjectRoleGrant revocation

Revocation:

- verifies an effective Project Manager scope for the project;
- prevents self-revocation through the admin operation when grant ownership equals acting actor;
- locks the active grant;
- sets status/revocation fields;
- appends audit and authority-invalidation events;
- invokes or schedules consuming lifecycle reconciliation.

For review leases, `WS-REV-001` grant-revocation behavior controls the lease transition.

18.3 Identity-link revocation

An Access Administrator may revoke an identity link only with a reason.

Effects are immediate for future requests. Existing request transactions that already completed authorization may finish; no new request may build a valid AuthorizationContext from the revoked link.

18.4 Identity-link reactivation

Reactivation:

- requires an effective Access Administrator;
- requires the linked ActorProfile not be deactivated;
- uses the same ActorIdentityLink row;
- records reason and audit event;
- does not modify roles or project grants.

19. Bootstrap Access Administrator

19.1 Purpose

The first Access Administrator cannot be created through an endpoint that already requires an Access Administrator.

19.2 Bootstrap sequence

1. The initial human signs in normally and calls `/v1/actors/me`.
2. Workstream creates the human ActorProfile and ActorIdentityLink.
3. An authorized deployment operator runs the Workstream-local bootstrap management operation with the exact ActorProfile ID.
4. The operation verifies that no active Access Administrator exists.
5. It creates one system-scoped `access_administrator` AdminRoleGrant.
6. `granted_by` references the fixed Workstream system bootstrap actor.
7. It appends `InitialAccessAdministratorBootstrapped` audit evidence.
8. It refuses every later invocation once an active Access Administrator exists.

19.3 Bootstrap prohibitions

- No public bootstrap HTTP endpoint exists.
- No shared bootstrap bearer secret exists.
- No manual SQL insert is part of the supported process.
- The operation cannot bootstrap an unknown ActorProfile.
- The operation cannot bootstrap a service actor.
- Additional Access Administrators are created through the normal grant API.

20. Core Operations

20.1 Read/update own profile

An active human may read their ActorProfile and update only permitted display fields.

The actor cannot change:

- actor kind;
- status;
- issuer or subject;
- roles or grants;

- provisioning method;
- audit fields.

20.2 Suspend actor

The transaction MUST:

1. authorize `actor.profile.suspend`;
2. lock the target ActorProfile;
3. require status `active`;
4. prohibit self-suspension;
5. enforce final Access Administrator safety;
6. set suspension fields;
7. append audit and invalidation events;
8. commit.

20.3 Reactivate actor

The transaction MUST:

1. authorize `actor.profile.reactivate`;
2. lock the ActorProfile;
3. require status `suspended`;
4. set status active and reactivation audit fields/event;
5. preserve separately revoked grants;
6. commit.

20.4 Deactivate actor

The transaction MUST:

1. authorize `actor.profile.deactivate`;
2. lock the ActorProfile;
3. require status active or suspended;
4. prohibit self-deactivation;
5. enforce final Access Administrator safety;
6. set terminal deactivation fields;
7. append audit and invalidation events;
8. commit.

20.5 Issue AdminRoleGrant

The transaction MUST:

1. authorize `admin_role.grant`;
2. lock the target ActorProfile;
3. require an active human target;
4. prohibit self-grant;

5. validate role/scope compatibility;
6. validate the referenced project when project-scoped;
7. verify no identical active grant exists;
8. insert the immutable grant;
9. append audit event;
10. commit.

20.6 Issue ProjectRoleGrant

The transaction MUST:

1. authorize `project.role_grant.manage` for the exact project;
2. lock the target ActorProfile and project active-grant selector;
3. require an active human target;
4. prohibit self-grant;
5. create the immutable qualification snapshot;
6. revoke an existing active grant only when the request explicitly replaces it;
7. create the new manual ProjectRoleGrant;
8. append audit and invalidation events where replacement occurred;
9. commit.

---

21. API Contract

All endpoints use the independent `/v1` API namespace. This does not change the product release from Workstream v0.1.

21.1 Current actor

```
GET   /v1/actors/me
PATCH /v1/actors/me
GET   /v1/actors/me/authorization-context?project_id={project_id}
```

PATCH request:

```
{
  "display_name": "Abiola Adeshina",
  "contact_email": "optional@example.com"
}
```

Authorization-context response:

```
{
  "actor_profile_id": "uuid",
  "status": "active",
  "domains": ["contributor", "admin"],
  "admin_role_grants": [],
  "project_role_grant": null,
  "capabilities": []
}
```

Capabilities are computed server-side from a registered allowlist. Clients cannot submit an arbitrary permission to be evaluated.

21.2 Actor administration

```
GET  /v1/admin/actors
GET  /v1/admin/actors/{actor_profile_id}
POST /v1/admin/actors/{actor_profile_id}/suspend
POST /v1/admin/actors/{actor_profile_id}/reactivate
POST /v1/admin/actors/{actor_profile_id}/deactivate
```

State-change request:

```
{
  "reason": "Required administrative reason"
}
```

21.3 Identity-link administration

```
GET  /v1/admin/actors/{actor_profile_id}/identity-link
POST /v1/admin/identity-links/{identity_link_id}/revoke
POST /v1/admin/identity-links/{identity_link_id}/reactivate
```

No endpoint adds a second human identity link in v0.1.

21.4 Service actors

```
POST /v1/admin/service-actors
GET  /v1/admin/service-actors
GET  /v1/admin/service-actors/{actor_profile_id}
```

21.5 Admin-role grants

```
POST /v1/admin-role-grants
GET  /v1/admin-role-grants
GET  /v1/actors/{actor_profile_id}/admin-role-grants
POST /v1/admin-role-grants/{grant_id}/revoke
```

Create request:

```
{
  "actor_profile_id": "uuid",
  "role": "project_manager",
  "scope_type": "project",
  "scope_project_id": "uuid",
  "reason": "Assigned to manage the project"
}
```

21.6 Project-role grants

```
POST /v1/projects/{project_id}/role-grants
GET  /v1/projects/{project_id}/role-grants
GET  /v1/projects/{project_id}/role-grants/{grant_id}
POST /v1/projects/{project_id}/role-grants/{grant_id}/revoke
```

Create request:

```
{
  "contributor_id": "uuid",
  "role": "reviewer",
  "grant_method": "manual",
  "grant_reason": "Qualified project reviewer",
  "qualification": {
    "skills_snapshot": {},
    "reputation_snapshot": {"status": "unavailable"},
    "prior_project_work_refs": [],
    "external_expertise_refs": []
  },
  "replace_active_grant": false
}
```

21.7 Permission catalog

```
GET /v1/authorization/permissions
GET /v1/authorization/admin-role-definitions
```

These endpoints return registered definitions. They do not expose or accept arbitrary dynamic policy code.

22. Error Contract

Errors MUST use the Workstream structured error envelope.

HTTP	Code	Condition
400	`invalid_request`	Malformed or incompatible request fields
401	`missing_token`	No bearer token
401	`invalid_token`	Signature, issuer, audience, time, or token format invalid
401	`invalid_token_claims`	Required verified claim missing or invalid
401	`identity_verification_unavailable`	Token cannot be safely verified
403	`required_scope_missing`	Coarse Workstream token scope missing
403	`unsupported_subject_kind`	Agent or Space token in v0.1
403	`service_actor_not_provisioned`	Unknown service subject
403	`identity_link_revoked`	External identity link revoked
403	`actor_suspended`	Actor is suspended
403	`actor_deactivated`	Actor is deactivated
403	`permission_not_granted`	No effective role/grant supplies the permission
403	`scope_not_authorized`	Admin role does not cover the project
403	`self_grant_forbidden`	Actor attempted to grant themselves authority
403	`self_role_revoke_forbidden`	Actor attempted to revoke own admin grant
403	`resource_guard_denied`	Global or consuming resource guard failed
404	`actor_not_found`	Authorized admin lookup target absent
404	`grant_not_found`	Authorized lookup target absent
404	`resource_not_found`	Resource absent or concealed by authorization policy
409	`actor_already_suspended`	Suspension replay conflicts with current state

HTTP	Code	Condition
409	`actor_not_suspended`	Reactivation target is not suspended
409	`actor_deactivated_terminal`	Transition attempted from deactivated state
409	`last_access_administrator`	Operation would remove final Access Administrator
409	`admin_role_grant_exists`	Identical effective grant already active
409	`project_role_grant_exists`	Active project grant exists and replacement not requested
409	`identity_link_conflict`	Issuer subject already maps to another ActorProfile
409	`resource_project_mismatch`	Related resources belong to different projects
409	`idempotency_mismatch`	Idempotency key reused with different request
422	`invalid_role_scope`	Role cannot use requested scope
422	`invalid_project_role`	Project contributor role invalid
422	`qualification_snapshot_invalid`	Required snapshot shape/ownership invalid

Unauthorized list and read endpoints MUST not reveal hidden records through total counts or distinct error messages.

23. Database Constraints and Indexes

Core invariants MUST be enforced in PostgreSQL.

23.1 Required constraints

```
ActorIdentityLink:
  UNIQUE(issuer, subject)
  UNIQUE(actor_profile_id)

AdminRoleGrant:
  partial UNIQUE(actor_profile_id, role)
    WHERE status = 'active' AND scope_type = 'system'
  partial UNIQUE(actor_profile_id, role, scope_project_id)
    WHERE status = 'active' AND scope_type = 'project'

ProjectRoleGrant:
  partial UNIQUE(project_id, contributor_id)
    WHERE status = 'active'

ProjectRoleQualificationSnapshot:
  immutable after insert

AuthorityControl:
  PRIMARY KEY(id)
  CHECK(id = 1)
```

Required check constraints:

```
ActorProfile:
  actor_kind IN ('human', 'service')
  status IN ('active', 'suspended', 'deactivated')
  automatic_first_access => actor_kind = 'human'
  manual_service_provisioning => actor_kind = 'service'

ActorIdentityLink:
  status IN ('active', 'revoked')
  subject_kind IN ('human', 'service')

AdminRoleGrant:
  system scope => scope_project_id IS NULL
  project scope => scope_project_id IS NOT NULL
  access_administrator/operator => system scope

ProjectRoleGrant:
  role IN ('submitter', 'reviewer', 'both')
  status IN ('active', 'revoked')
  manual => automation_policy_ref IS NULL
```

Foreign keys MUST prevent project/snapshot/contributor mismatches. Composite foreign keys MUST be used where necessary.

23.2 Required indexes

```
ActorProfile(status, actor_kind)
ActorProfile(last_seen_at)
ActorIdentityLink(issuer, subject, status)
AdminRoleGrant(actor_profile_id, status)
AdminRoleGrant(role, scope_type, scope_project_id, status)
ProjectRoleGrant(project_id, contributor_id, status)
ProjectRoleGrant(contributor_id, status)
ProjectRoleQualificationSnapshot(project_id, contributor_id, captured_at)
AuditEvent(actor_profile_id, occurred_at)
AuditEvent(project_id, occurred_at)
```

All timestamps are stored in UTC. Authorization expiry/time comparisons use database time.

24. Transaction and Concurrency Requirements

24.1 First-access race

Two concurrent requests for one new `(issuer, subject)` result in one ActorProfile and one ActorIdentityLink.

24.2 Duplicate AdminRoleGrant race

Two concurrent identical grant requests result in one active grant. An exact idempotent replay returns the existing grant; another request returns `admin_role_grant_exists`.

24.3 Duplicate ProjectRoleGrant race

Two concurrent project-grant requests for one contributor/project result in one active grant. Replacement requires locking the active grant selector and an explicit replacement request.

24.4 Last Access Administrator race

Concurrent operations that would revoke, suspend, or deactivate Access Administrators MUST lock a common authority-guard row or equivalent database-protected selector. At least one effective active Access Administrator must remain after commit.

Counting without a shared lock is insufficient.

24.5 Grant versus authorization

Authorization loads effective grants inside the command transaction for sensitive mutations.

The outcome may be:

- authorization locks/observes the active grant before revocation commits and the command completes; or
- revocation commits first and the command is denied.

A router-level role check performed before the transaction MUST be revalidated inside the application transaction for sensitive mutations.

24.6 Suspension versus active request

A request already authorized and committed before suspension remains valid history. Suspension prevents later authorization. Long-running workers MUST re-resolve current ActorProfile and grant state before committing an actor-attributed mutation.

24.7 Idempotency

The following mutations require an idempotency key:

- service actor creation;
- AdminRoleGrant creation/revocation;
- ProjectRoleGrant creation/revocation;
- actor suspend/reactivate/deactivate;
- identity-link revoke/reactivate.

Same key and same canonical request returns the existing result. Same key and different request returns `idempotency_mismatch`.

25. Audit Events

All events are append-only.

Identity and profile

- `ActorProfileProvisioned`
- `ServiceActorProvisioned`
- `ActorIdentityLinked`
- `ActorIdentityLinkRevoked`
- `ActorIdentityLinkReactivated`
- `ActorProfileSuspended`
- `ActorProfileReactivated`
- `ActorProfileDeactivated`

Admin authority

- `InitialAccessAdministratorBootstrapped`
- `AdminRoleGrantIssued`
- `AdminRoleGrantRevoked`

- `AdminRoleGrantIssueDenied`
- `LastAccessAdministratorOperationDenied`

Project contributor authority

- `ProjectRoleQualificationSnapshotCaptured`
- `ProjectRoleGrantIssued`
- `ProjectRoleGrantReplaced`
- `ProjectRoleGrantRevoked`

Authorization

- `SensitiveAuthorizationAllowed`
- `SensitiveAuthorizationDenied`
- `AuthorityInvalidationRequested`

Every event includes:

- event ID;
- event version;
- occurred-at database timestamp;
- acting ActorProfile or system actor;
- target ActorProfile;
- effective grant ID used by the acting actor;
- project ID where applicable;
- target grant/link ID;
- reason;
- correlation ID;
- idempotency key where applicable;
- before and after state for mutable records.

Tokens, raw JWTs, secrets, and full identity-provider payloads MUST NOT be stored in audit events.

---

26. Authorization Service Interface

The application layer MUST expose one internal interface equivalent to:

```
class AuthorizationService:
    async def build_context(
        self,
        verified_token: VerifiedIssuerToken,
        request_id: UUID,
        correlation_id: UUID,
    ) -> AuthorizationContext: ...

    async def authorize(
        self,
        context: AuthorizationContext,
        permission: Permission,
        resource: ResourceContext,
        *,
        uow: UnitOfWork,
    ) -> AuthorizationDecision: ...

    async def require(
        self,
        context: AuthorizationContext,
        permission: Permission,
        resource: ResourceContext,
        *,
        uow: UnitOfWork,
    ) -> AuthorizationDecision: ...
```

`require` raises the stable authorization error for a denied decision.

`Permission` must be a registered enum/value object, not an unchecked free-form string from a client.

`ResourceContext` includes the canonical resource type, ID, resolved project ID, owner/creator actor ID where required, and current lifecycle state.

---

27. FastAPI Integration

27.1 Dependencies

Implement reusable dependencies equivalent to:

```
get_verified_issuer_token
get_or_provision_actor_context
require_active_actor
require_permission(permission, resource_loader)
```

27.2 Router rules

Routers MUST:

- validate the request contract;
- resolve the AuthorizationContext;
- invoke one application command/query;
- return the stable response/error envelope.

Routers MUST NOT:

- query role tables directly;
- perform permission unions;
- infer project ownership from request JSON;
- implement self-review or task-ban checks locally;
- forward the human token to another service.

27.3 Worker rules

A worker executing an actor-attributed command MUST load the current ActorProfile and grant state before committing. The actor context serialized into a job is evidence of the request, not current authority.

System-owned recovery workers use a fixed Workstream system actor and explicit system permission, not a fabricated human administrator.

28. Security Requirements

1. Deny by default.
2. Trust only configured Identity Issuer algorithms, issuer, audience, and JWKS.
3. Never accept Workstream role claims from the token as canonical.
4. Never use email, display name, skill, or reputation as a direct authorization key.
5. Never auto-provision service, agent, or Space subjects.
6. Never let an actor grant themselves authority.
7. Never permit loss of the final Access Administrator.
8. Never expose token/JWKS material in application logs.
9. Never cache authorization decisions across requests.
10. Revalidate authority inside sensitive mutation transactions.
11. Use constant, registered permission identifiers.
12. Enforce project ownership in both authorization service and database relationships.
13. Rate-limit first-access provisioning and admin mutation endpoints through the existing API control layer.
14. Protect bootstrap operation as a local deployment operation.
15. Encrypt production connections and managed storage.
16. Record reasons for every suspension, deactivation, role grant, and revocation.

29. Privacy and Disclosure

- Actor search/list results expose only fields required for the caller's role.
- Access Administrators may view identity-link metadata but not raw tokens or provider credentials.
- Audit Authority receives minimal actor attribution, not unrestricted profile data.
- Project Managers may search contributors needed for their project grant workflow but may not enumerate unrelated private project activity.
- Finance Authority receives contribution and award fields required for compensation, not unrelated identity/profile data.
- Unauthorized project records are omitted before counts and pagination cursors are calculated.
- Profile contact email is never an authorization key and is excluded from events unless explicitly required by a notification adapter.

30. Observability

30.1 Logs and traces

Include when applicable:

- correlation ID;
- request ID;
- ActorProfile ID;
- actor kind/status;
- permission identifier;
- project/resource identifiers;
- matched grant IDs;
- decision allow/deny and denial code;
- token issuer and hashed/token-safe subject reference.

Do not include bearer tokens, raw JWT claims, secrets, or unnecessary personal data.

30.2 Metrics

Required metrics:

```
actor_first_access_total{result}
actor_first_access_conflict_total
actor_status_total{status,kind}
identity_link_total{status,kind}
token_verification_total{result}
jwks_refresh_total{result}
authorization_decision_total{permission,result,denial_code}
admin_role_grant_total{role,scope,status}
project_role_grant_total{role,status}
authority_invalidation_total{reason}
bootstrap_attempt_total{result}
```

30.3 Alerts

Alert on:

- sustained token verification failure;
- JWKS refresh failure without valid cache;
- repeated bootstrap attempts after initialization;
- repeated last-access-admin operation attempts;
- unusual admin-role grant/revoke rate;
- authority invalidation reconciliation backlog;
- actor provisioning unique-conflict rate above expected concurrent first access.

31. Conformance Tests

31.1 Token tests

- Valid issuer token with Workstream audience succeeds.

- Wrong issuer fails.
- Wrong audience fails.
- Unknown algorithm fails.
- Invalid signature fails.
- Expired/not-yet-valid token fails.
- Missing `sub`, `jti`, or `subject_kind` fails.
- Missing coarse scope fails.
- Unknown `kid` triggers one refresh then fails safely.
- Better Auth token is not accepted as final product token.

31.2 First-access tests

- New human token creates one ActorProfile and one ActorIdentityLink.
- Created human is Contributor-domain only.
- Created human has no AdminRoleGrant or ProjectRoleGrant.
- Repeated access returns the same ActorProfile.
- Concurrent first access creates no duplicate or orphan.
- Unknown service token creates nothing and is denied.
- Agent/Space tokens create nothing and are denied.

31.3 Actor-state tests

- Access Administrator suspends another actor with reason.
- Suspended actor cannot mutate business state.
- Reactivation restores non-revoked grants.
- Deactivation is terminal.
- Historical records remain linked.
- Access Administrator cannot suspend/deactivate self.
- Final Access Administrator cannot be suspended/deactivated.

31.4 AdminRoleGrant tests

- Only Access Administrator can issue/revoke.
- Self-grant is denied.
- Self-revoke is denied.
- Role/scope compatibility is enforced.
- Duplicate active grant is prevented by database constraint.
- Project-scoped role cannot act in another project.
- System-scoped role receives only its own permission group.
- Revocation takes effect without a new issuer token.

31.5 ProjectRoleGrant tests

- Only Project Manager covering the project can grant/revoke.

- Project Manager cannot grant themselves contributor authority.
- Snapshot is mandatory and immutable.
- Missing reputation may be recorded unavailable.
- Submitter grant cannot review.
- Reviewer grant cannot claim submission tasks.
- Both grant supplies union but not self-review bypass.
- Admin role alone cannot submit or review.
- Duplicate active project grant is prevented.
- Replacement creates a new grant and revokes old history.
- Revocation is visible on next request.

31.6 Permission matrix tests

For every permission identifier, create table-driven tests proving:

- each allowed role/scope combination;
- each denied role combination;
- project-scope boundary;
- system scope does not create superuser authority;
- explicit resource guard overrides an allowed permission candidate;
- unauthorized list counts do not leak hidden records.

31.7 Bootstrap tests

- Bootstrap succeeds only for an existing active human ActorProfile.
- Bootstrap succeeds only when zero active Access Administrators exist.
- Bootstrap creates system-scoped Access Administrator.
- Later bootstrap attempt fails and is audited.
- Service actor cannot be bootstrapped.
- No public bootstrap endpoint exists.

31.8 Concurrency tests

- Concurrent human first access leaves one profile/link.
- Concurrent identical AdminRoleGrant leaves one active grant.
- Concurrent ProjectRoleGrant leaves one active grant.
- Concurrent final-admin suspension/revocation attempts leave at least one effective Access Administrator.
- Grant revocation versus sensitive command produces one serializable authority outcome.

31.9 Service actor tests

- Access Administrator can provision a service actor.
- Service actor has no Contributor domain.
- Service actor cannot receive AdminRoleGrant or ProjectRoleGrant.

- Active service binding is still required by consuming adapter endpoint.
- Suspended service actor cannot call callback/integration endpoint.

32. Live API Drill

The live drill MUST use supported Workstream APIs and the bootstrap operation. Direct database changes invalidate the drill.

32.1 Drill sequence

1. Start Workstream with no ActorProfiles or Access Administrators.
2. Human A presents a valid issuer token and calls `/v1/actors/me`.
3. Prove Human A has one active ActorProfile, Contributor domain, and no roles/grants.
4. Run the one-time bootstrap operation for Human A.
5. Prove Human A now has system-scoped `access_administrator` only.
6. Human B signs in and is provisioned as a default Contributor.
7. Human A grants Human B system-scoped `project_manager`.
8. Human B creates or manages Project P.
9. Human C signs in and is provisioned as a default Contributor.
10. Human B captures a qualification snapshot and grants Human C `submitter` for Project P.
11. Prove Human C can access submitter capability for Project P but cannot access review queue capability.
12. Human D signs in.
13. Human B grants Human D `reviewer` for Project P.
14. Prove Human D can access reviewer capability for Project P but cannot claim submitter work.
15. Prove Human B cannot review by Project Manager role alone.
16. Human A grants Finance Authority to Human E for Project P.
17. Prove Human E can manage compensation-policy capability but cannot manage ProjectRoleGrant or review.
18. Human A grants Audit Authority to Human F for Project P.
19. Prove Human F can read authorized audit views but no mutation.
20. Revoke Human D's reviewer grant and prove the same issuer token no longer authorizes review capability.
21. Suspend Human C and prove all business mutations are denied.
22. Reactivate Human C and prove the non-revoked submitter grant is effective again.
23. Attempt self-grant, self-admin-role revoke, cross-project access, and final Access Administrator removal; prove all are denied and audited.
24. Provision one service actor and prove an unknown service token is denied while the provisioned service actor resolves.
25. Export the complete audit chain and database constraint evidence.

32.2 Drill evidence

The report MUST include:

- request and correlation IDs;
- relevant response bodies and error codes;
- ActorProfile/IdentityLink identifiers;
- role/grant identifiers and scope;
- database constraint checks;
- audit events;
- proof that no duplicate ActorProfile was created;
- proof that revocation used the same unexpired issuer token;
- proof that admin roles did not substitute for ProjectRoleGrant;
- proof that no direct database edit occurred except the supported bootstrap operation.

33. Implementation Delivery Order

Phase 1: Token verification and request context

- issuer/JWKS configuration;
- VerifiedIssuerToken contract;
- token verification dependency;
- safe JWKS cache/refresh;
- structured authentication errors.

Exit: token conformance tests pass.

Phase 2: ActorProfile and identity link

- migrations and constraints;
- first-human-access transaction;
- service subject denial;
- actor resolver;
- self-profile APIs.

Exit: first-access and concurrency tests pass.

Phase 3: Bootstrap and AdminRoleGrant

- system bootstrap actor;
- one-time bootstrap operation;
- AdminRoleGrant schema;
- five role definitions and scope compatibility;
- grant/revoke APIs;
- last-access-admin concurrency guard.

Exit: bootstrap and admin-role conformance tests pass.

Phase 4: Actor state and identity-link administration

- suspend/reactivate/deactivate;
- link revoke/reactivate;
- service actor provisioning;
- authority invalidation events.

Exit: actor-state and service-actor tests pass.

Phase 5: ProjectRoleGrant

- qualification snapshot;
- project grant schema/constraints;
- Project Manager authority;
- create/replace/revoke APIs;
- project permission candidates.

Exit: project-grant and concurrency tests pass.

Phase 6: Authorization service

- permission registry;
- role matrices;
- scope resolver;
- ResourceContext loaders;
- global resource guards;
- FastAPI permission dependencies;
- worker revalidation.

Exit: table-driven permission-matrix tests pass.

Phase 7: Audit, observability, and live drill

- all audit events;
- metrics/traces;
- denied-decision evidence;
- live API drill;
- coding-agent evidence pack.

Exit: definition of done passes and `WS-REV-001` may begin.

---

34. Definition of Done

This specification is complete only when:

- Workstream verifies only final issuer-owned tokens and JWKS.
- Every valid first-time human subject creates exactly one active ActorProfile and identity link.
- Default human actors are Contributor-domain members with no project or admin authority.
- Unknown service, agent, and Space subjects are not auto-provisioned.

- Service actor provisioning is controlled and audited.
- The five AdminRoleGrants and scope rules are implemented.
- ProjectRoleGrant is project-scoped, manual, snapshot-backed, and revocable.
- Admin roles never substitute for submitter/reviewer ProjectRoleGrant.
- Permission catalog and role matrices are centralized and table-tested.
- Authorization resolves canonical resource project/ownership and applies resource guards.
- Role revocation takes effect with the same still-valid issuer token.
- Actor suspension/deactivation invalidates authority without deleting history.
- Final Access Administrator safety is database/concurrency proven.
- All required constraints and indexes exist in PostgreSQL.
- Every sensitive authority change is auditable.
- The live API drill completes with no unsupported database edit.
- Existing Workstream intake tests remain green.
- The implementation evidence pack distinguishes implemented, tested, and live-proven behavior.

---

35. Explicitly Out of Scope

- Automated ProjectRoleGrant issuance from skills or reputation thresholds.
- Reputation scoring or reputation-policy administration.
- Delegated agent access to Workstream.
- Space subjects and organization membership.
- Multiple Identity Issuer subjects linked to one Workstream ActorProfile.
- Subject merge or split.
- Dynamic administrator roles created through the API.
- User-authored permission policies.
- Attribute-policy language or general policy engine.
- Adjudication above human review.
- Payment provider, points ledger, wallet, or settlement implementation.
- Direct human access to Flow Node.

These may be additive future specifications. They MUST NOT be inferred into v0.1 code.

---

36. Coding-Agent Handoff Rules

The coding agent MUST:

1. read this specification completely before modifying code;
2. inspect existing FastAPI auth, actor, role, project, audit, and migration structures;
3. extend existing canonical objects instead of creating parallel models;
4. preserve the existing intake lifecycle and tests;
5. implement phases in the stated order;

6. use PostgreSQL for constraint and concurrency proof;
7. centralize all permission evaluation in AuthorizationService;
8. stop and report a normative conflict rather than inventing a role or permission;
9. produce the live drill and implementation evidence pack;
10. begin `WS-REV-001` only after this specification's definition of done is satisfied.

The coding agent MUST NOT:

- add a second ActorProfile for administrators;
- add a single mutable `role` column to ActorProfile;
- accept role claims from the Identity Issuer token as product authority;
- auto-provision service, agent, or Space actors;
- grant project authority from skills or reputation automatically;
- allow self-grant or loss of the final Access Administrator;
- implement permissions separately in routers;
- cache role decisions across requests;
- remove authority history;
- introduce unrelated products or infrastructure.